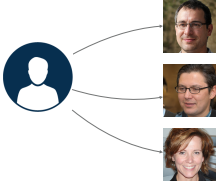


NESNEYE YÖNELİK PROGRAMLAMA

Sınıflar ve Nesneler

SINIF VE NESNE KAVRAMLARI

- Sınıf
 - Nesneye yönelik programlama
 - Struct benzeri
- Nesne
 - Örnek



Sınıflar, oluşturulacak nesnelerin bir planı veya prototipidir. Nesnelerin sahip olacağı özellikleri ve aksiyonları tanımlarlar.

Bir sınıf, tıpkı bir ev planı gibi düşünülebilir: plan kâğıt üzerindedir, fiziksel olarak bir yer kaplamaz; ama o plandan istediğiniz kadar gerçek ev inşa edebilirsiniz.

Sınıf tanımı da bellekte yer kaplamaz — yalnızca bir şablon olarak durur.

Bellekte yer ancak o şablondan bir nesne oluşturulduğunda ayrılır.

Sınıflar C'deki structure'lara benzeyen ve nesneye yönelik programlama dillerine özgü olan bir yapıdır.

C'deki struct yalnızca veri tutabilirken, sınıflar hem veri hem de o veriyle ilgili işlemleri (metotları) bir arada barındırır. Java, C++ ve Python bu yapıyı destekleyen nesneye yönelik dillerdir.

Nesneler ise sınıfların sahip olduğu özelliklerin değerlerini ve aksiyonların nasıl olacağını içerirler. Sınıflardan oluşturulmuş birer örnektirler.

Aynı sınıftan birden fazla nesne oluşturulabilir ve her nesne kendi bağımsız veri kopyasını taşır. Örneğin Araba sınıfından oluşturulan "BMW" nesnesi ile "Toyota" nesnesi aynı özelliklere (marka, yıl, hız) sahiptir; ancak bu özelliklerin değerleri birbirinden tamamen bağımsızdır. Birinin değeri değiştirildiğinde diğeri etkilenmez.

YAPILAR (STRUCT)

- > Yapı tanımı
 - > Özellikler
 - > Kullanıcı tanımlı [değişkenler](#)
 - > Metotlar
 - > Yazdırma
 - > Java

Yapılar farklı veri tiplerinin bir arada saklanabildiği veri türleridir.

Farklı primitif değişkenler bir araya getirilerek kullanıcı tanımlı yapılar oluşturulabilmektedir.

Örneğin bir Öğrenci yapısı; String türünde ad, int türünde numara ve double türünde not gibi birbirinden farklı tipleri tek çatı altında bir araya getirebilir. Bu sayede ilgili verileri ayrı ayrı değişkenlerde tutmak yerine tek bir birim olarak taşımak mümkün olur.

Yapılar veri tiplerini içerirler fakat kendilerine ait metot saklamazlar.

Yapılar veri üzerinde işlem yapan fonksiyonları kendi bünyelerinde barındıramazlar. Doğrulama, biçimlendirme veya hesaplama gibi işlemler için yapının dışında ayrı fonksiyonlar yazmak gerekir. Bu durum kodun dağınıklaşmasına yol açar.

Örneğin yapıların içerisinde bulunan özelliklerin gösterilmesi için yazdırma fonksiyonları kullanılamaz.

Herhangi bir yazdırma işlemi için harici bir fonksiyon tanımlanması gerekir.

Yapıya yeni bir alan eklendiğinde veya mevcut bir alan kaldırıldığında, bu harici fonksiyonların da güncellenmesi zorunlu hale gelir. Bu bakım yükü, proje büyüdükçe ciddi bir sorun oluşturur.

Ayrıca Java'da structure kullanılamamaktadır.

Java'da struct dil seviyesinde desteklenmez. Java'da bu ihtiyacı karşılamak için sınıflar kullanılır; ancak tüm alanların public yapılması OOP ilkelerine aykırıdır ve aşağıdaki slaytlarda ele alınan dezavantajların aynısını doğurur.

YAPILAR (STRUCT)

- > C++'ta mevcut
- > Değer kopyalama gerçekleştirilir
- > Stack
 - > Eğer struct elemanları referans saklamıyorsa ve çok büyük değilse

```
#include <iostream>

typedef struct yapisi {
    int saklanacakIlkAlan;
    double ikinciAlan;
    std::string buradaStringVar;
}Yapisi;

int main(int argc, const char * argv[]) {
    Yapisi y;
    y.saklanacakIlkAlan = 1;
    y.ikinciAlan = 1.1;
    y.buradaStringVar = "Daha iyi";
    return 0;
}
```

Yapılar C++ dilinde sınıfların yanında tanımlanabilirler.

C++'ta hem struct hem de class anahtar sözcükleri mevcuttur. Aralarındaki tek resmi fark varsayılan erişim düzeyidir: struct'ta tüm üyeler varsayılan olarak public, class'ta ise private'tır. Bunun dışında her ikisi de metot, constructor ve kalıtımı destekler.

Genellikle sınıfların aksine heap yerine stack'te saklanırlar.

Stack belleği CPU tarafından otomatik yönetilir; bir fonksiyon çağrısı bittiğinde stack üzerindeki tüm yerel değişkenler ve yapılar otomatik olarak temizlenir. Bu nedenle struct'lar için ayrıca delete çağırmanıza gerek kalmaz.

Yapının çok büyük olması veya referans değerleri taşıması durumlarında ise heap'e alınabilirler.

Büyük yapılar stack'e kopyalanırken ciddi bir performans maliyeti oluşturabilir.

Ayrıca heap üzerindeki bir kaynağa işaret eden pointer gibi referans değerleri taşıyan yapıların ömrünün dikkatli yönetilmesi gerekir.

Stack'te saklandıkları için değer kopyalama gerçekleştirilir.

Bir struct değişkenini başka bir değişkene atadığınızda veya bir fonksiyona parametre olarak geçirdiğinizde, tüm alanların değeri kopyalanır. Bu, iki değişkenin birbirinden bağımsız birer kopya olduğu anlamına gelir; birini değiştirmek diğerini etkilemez. Sınıf nesnelerinde ise (heap'te tutulduğundan) referans kopyalanır: her iki değişken de aynı nesneyi gösterir.

YAPILAR - SINIFLAR

```
class kisi{
    public String ad;
    public int yas;
}

public class Main {
    public static void main(String[] args){
        kisi k = new kisi();
        k.ad = "Emir";
        k.yas = 25;
        System.out.println(k);
    }
}
```

net.emirozturk.kisi@77459877

```
System.out.println(k.ad + " " + k.yas);
```

```
typedef struct kisi{
    char* ad;
    int yas;
}kisi;

int main(int argc, char* argv[]) {
    kisi k;
    k.ad = "Emir";
    k.yas = 25;
    printf("k.ad: %s\n", k.ad);
    return 0;
}
```

```
std::cout << k.ad << " " << k.yas << std::endl;
```

Yapılar kullanılamasa da Java'da sınıflar, yapıların sunduğu şekilde erişim için kullanılabilirler. Nesneye yönelik programlama kuralları dışına çıkılması ile birlikte sınıf alanları struct gibi tanımlanabilmektedir.

Bunun için tüm alanların public yapılması yeterlidir. Böylece nesne.yas = 25 gibi doğrudan atama C'deki struct erişimiyle aynı sözdizimini taklit eder. Ancak bu yaklaşım, veri doğrulamayı ve kapsüllemeyi tamamen ortadan kaldırdığı için OOP'un sunduğu tüm avantajlardan vazgeçmek anlamına gelir.

Ekrana çıktı gösterme aşamasında Java'da her nesne Object'ten geldiği için toString metodu ile yazdırılabilmektedir fakat struct'larda bu metod olmadığı için yazdırma işlemi gerçekleştirilemez.

Java'da tüm sınıflar Object sınıfından türer. Object sınıfının toString() metodu varsayılan olarak sınıf adı ve bellek adresi döndürür; bunu @Override ile ederek anlamlı bir çıktı üretebilirsiniz. C'deki struct'larda ise böyle bir kalıtım zinciri yoktur; yazdırma her zaman harici bir fonksiyona ihtiyaç duyar.

YAPILAR - SINIFLAR

- Yapı içeriğinin doğruluğu
- Yapıyı kullanan fonksiyonların üzerindeki değişiklik

```
class kisi{
    public String ad;
    public int yas;
    public int no;
}
```

```
k.yas = 11121;
```

```
public class Main {
    public int sonIkIHane(kisi k){
        return k.no % 100;
    }
    public boolean ilkIkiBuyukMu(kisi k1, kisi k2){
        return sonIkIHane(k1) > sonIkIHane(k2);
    }
    public static void main(String[] args){
    }
}
```

Yapıların sınıflara göre bazı dezavantajları bulunmaktadır.

Veri girişi / erişimi açık olduğu için verinin geçerli girilip girilmediği garantisizdir (bir önceki slayttaki yaş değeri gibi).

Yapı alanlarına doğrudan erişildiğinde herhangi bir kontrol mekanizması devreye girmez. kisi.yas = -99 veya kisi.yas = 999 gibi anlamsız değerlerin atanmasının önüne geçilemez. Programın doğruluğu, o alanı kullanan her programcının dikkatine ve iyi niyetine bırakılmış olur. Ekip büyüdükçe veya zaman içinde veri tutarsızlıkları kaçınılmaz hale gelir.

Yapı değiştiğinde yapıyı gösteren ya da üstünde işlem yapan fonksiyonların

tamamının tekrar düzeltilmesi gerekecektir.

Bir yapıya yeni alan eklenmesi, mevcut bir alanın kaldırılması veya isminin değiştirilmesi durumunda, o yapıyı parametre olarak alan, döndüren ya da alanlarına erişen her fonksiyon manuel olarak güncellenmelidir. Bu bağımlılık zinciri, kodun bakımını zorlaştırır ve tek bir değişikliğin birçok noktada hataya yol açmasına zemin hazırlar. Sınıflarda ise bu tür değişiklikler büyük ölçüde sınıfın kendi içinde izole kalır.

YAPILAR - SINIFLAR

- Doğruluğun sağlanması
 - Erişim belirleyicileri
 - Erişim metotları
 - Yapı ile ilgili metotlar

Böyle bir durumun çözümü için değişkenlere dışarıdan erişimin kapatılması gerekmektedir.

private anahtar sözcüğü ile tanımlanan alanlar yalnızca sınıfın kendi metotlarından erişilebilir hale gelir. Dışarıdan nesne.yas = 999 gibi bir atama artık derleme hatası verir. Bu sayede geçersiz veri, sisteme girmeden engellenmiş olur.

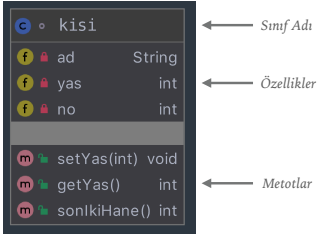
Ayrıca dışarıdan erişim sağlanamayan değişkenlere belirli kurallar dahilinde erişebilmek için metotlar gerekmektedir.

Bu metotlar getter (okuma) ve setter (yazma) olarak adlandırılır. Getter, private alanın değerini dışarıya döndürür. Setter ise yeni değer atar. Bu sayede alana erişim tamamen kontrol altına alınır.

Nesneyle ilgili tüm işlemlerin sınıfın kendi metotları olması, kodu daha okunabilir, bakımı daha kolay hale getirir. Bir nesnenin ne yapabileceği, o nesnenin sınıfına bakıldığında açıkça görülür; ayrı ayrı fonksiyon aramaya gerek kalmaz. Bu ilke nesneye yönelik programlamanın kapsülleme (encapsulation) ilkesinin özüdür.

SINIF BİLDİRİMİ

- Sınıflar
 - Sınıf ismi
 - Özellikler (değişkenler)
 - Metotlar (fonksiyonlar)



Sınıflar, verilerin erişim düzeyinin belirlenmesi desteğini sunar ve kendine ait metotları bulunduğu için erişime kapalı değişkenlerin erişim ayarları istenilen metotlar kullanılarak gerçekleştirilebilmektedir.

Sınıflar struct'ın sunduğu veri gruplama özelliğini korurken ona ek olarak iki kritik yetenek kazandırır: erişim kontrolü ve metot kapasitesi. private alan ile dışarıya kapalı tutulan veri, public setter/getter metotları aracılığıyla kontrollü biçimde kullanılabilir hale gelir.

Sınıflar, sınıf ismi, sınıfın içerisindeki özellikler ve metotlardan oluşmaktadır. Sınıf ismi PascalCase yazılır (her kelimenin baş harfi büyük: BankaHesabi, OgrenciKaydi). Özellikler (fields) genellikle private tanımlanır ve nesnenin durumunu temsil eder. Metotlar ise hem getter/setter gibi erişim metotlarını hem de nesnenin davranışını tanımlayan iş mantığı fonksiyonlarını kapsar.

SINIF BİLDİRİMİ (JAVA-CPP-PYTHON)

```
class SınıfAdı {  
    ErişimTürü DeğişkenTürü ad1;  
    ErişimTürü DeğişkenTürü ad2;  
    ErişimTürü DeğişkenTürü ad3;  
  
    ErişimTürü DönüşTipi İsim(Tür ad){  
        //İşlemler  
    }  
    ErişimTürü DönüşTipi İsim(){  
        //İşlemler  
    }  
}
```

```
class SınıfAdı {  
    ErişimTürü:  
    DeğişkenTürü ad1;  
    DeğişkenTürü ad2;  
    DeğişkenTürü ad3;  
  
    ErişimTürü:  
    DönüşTipi İsim(Tür ad){  
        //İşlemler  
    }  
    DönüşTipi İsim(){  
        //İşlemler  
    }  
}
```

```
class SınıfAdı:  
    #Değişken isimleri yapıcıda  
    #tanımlanır  
    def İsim(ad):  
        #İşlemler  
  
    def İsim():  
        #İşlemler
```

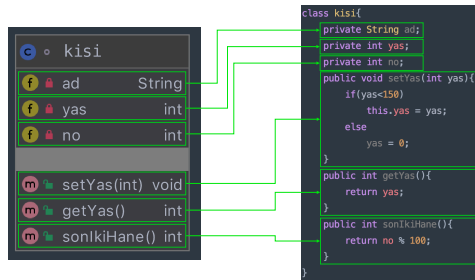
Java'da erişim türleri değişkenlerin ve metotların başında verilirken C++'ta ise etiket olarak verilmektedir. Python'da ise erişim belirleyicisi verilmemektedir.

Değişken tanımları da yapıcı içerisinde verilmektedir.

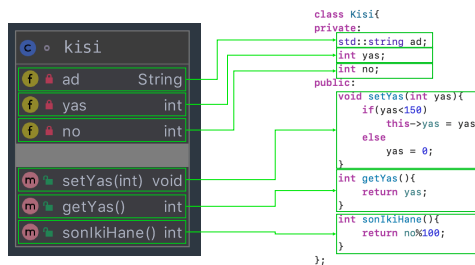
Java'da her üye kendi erişim belirleyicisini taşır: private int yas; şeklinde satır satır belirtilir. C++'ta ise private:, public:, protected: etiketleri bir blok başlatır; o etiketin altında tanımlanan tüm üyeler aynı erişim düzeyini alır. Python'da ise dil seviyesinde bir erişim belirleyici sözdizimi yoktur. Bunun yerine isimlendirme kuralı kullanılır: tek alt çizgi (_alan) dışarıdan erişimi zorlaştırır ancak tamamen

engelleyemez. Python'da ayrıca sınıf alanları genellikle init yapıcı metodu içinde self.alan = değer biçiminde tanımlanır; sınıf gövdesinde doğrudan alan bildirimi yapılmaz.

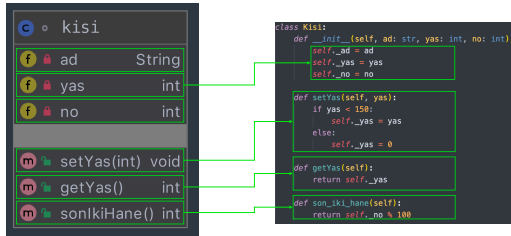
SINIF BİLDİRİMİ (JAVA)



SINIF BİLDİRİMİ (CPP)



SINIF BİLDİRİMİ (PYTHON)



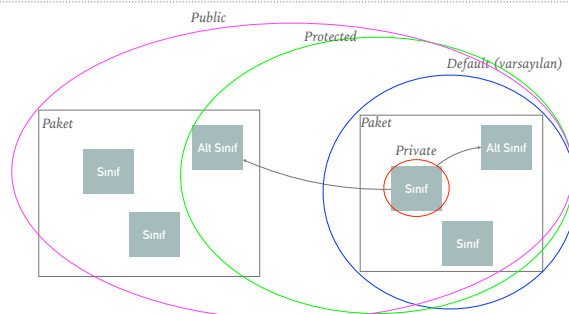
SINIF BİLDİRİMİ (PYTHON)



Python'da veri türlerinin belirtilmesi mümkündür. Aynı zamanda setter getter yazılabilmekle beraber Python'da erişim belirleyicileri aşılabildiğinden dolayı pratikte bu metotların kullanımı nadirdir.

Python 3.5 ile gelen tip anotasyonları (type hints) sayesinde marka: str veya yas: int gibi notasyonlar kullanılabilir. Ancak bu notasyonlar derleme zamanı kontrolü sağlamaz; program yine de yanlış tipte çalışır. IDE'ler ve mypy gibi statik analiz araçları bu bilgiyi kullanarak uyarı üretebilir.

ERİŞİM BELİRLEYİCİLERİ



Java'da dört farklı erişim belirleyicisi bulunmaktadır.

private: private yapılan özellik ve metotlar sadece sınıf içerisinde erişilebilmektedirler.

Sınıfın kendisi dışındaki hiçbir kod — alt sınıflar dahil — bu alanlara doğrudan erişemez. Erişim yalnızca aynı sınıf içinde tanımlanan metotlar aracılığıyla mümkündür.

default: herhangi bir erişim belirleyicisi belirtilmediğinde (default), erişim sınıfta ve paket içerisinde kalıtılmış alt sınıfta mümkündür. Eğer kalıtılan sınıf (alt sınıf) başka bir paket içerisinde ise bu değişkenlere bu sınıf üzerinden erişilemez.

Açıkça bir belirleyici yazılmadığında Java bu düzeyi otomatik atar; buna "paket-private" da denir. Aynı paket içindeki her sınıf bu üyelere ulaşabilir. Ancak farklı bir paketten extends ile türetilen bir alt sınıf, bu alanlara erişmeye çalıştığında derleme hatası alır.

protected: default erişim belirleyicisine ek olarak başka paketlerdeki alt sınıflardan da erişim mümkündür.

Kalıtım hiyerarşisinde üst sınıfın alt sınıflara sunmak istediği, ancak dışarıya tamamen açmak istemediği üyeler için kullanılır. Farklı paketteki bir alt sınıf protected alana extends ilişkisi sayesinde erişebilir; ancak o paketin dışındaki

ERİŞİM BELİRLEYİCİLERİ

Erişim Belirleyici	C++	Java	Python
Public	Her yerden erişilir	Her yerden erişilir	Anahtar kelime bulunmaz Her yerden erişilir
Private	Yalnızca sınıf içerisinden erişilir	Yalnızca sınıf içerisinden erişilir	.. ile tanımlananlar private kabul edilir Buna rağmen her yerden erişilir
Protected	Sınıf içerisinden ve türetilmiş sınıflardan erişilir	Sınıf içerisinden ve türetilmiş sınıflardan erişilir	.. ile tanımlananlar private kabul edilir Buna rağmen her yerden erişilir
Varsayılan	Private	Paket içi erişim	Anahtar kelime bulunmaz Her yerden erişilir

ERİŞİM BELİRLEYİCİLERİ

```
class kisi{
    public String ad;
    public int yas;
    public int no;
}

public class Main {
    public int sonKiHane(kisi k){
        return k.no % 100;
    }

    public boolean ikiKiYukulu(kisi k1,kisi k2){
        return sonKiHane(k1)>sonKiHane(k2);
    }

    public static void main(String[] args){
    }
}
```

```
class kisi{
    private String ad;
    private int yas;
    private int no;
    public void setYas(int yas){
        if(yas<150)
            this.yas = yas;
        else
            yas = 0;
    }
    public int getYas(){
        return yas;
    }
    public int sonKiHane(){
        return no % 100;
    }
}
```

Sınıflar tanımlanırken değişkenler soyutlanır.

Soyutlama burada şu anlama gelir: dışarıdan bakıldığında "Araba sınıfının marka alanı var" bilgisi gizlenir; yalnızca "markayı alabilirsiniz ve markayı belirli kurallarla değiştirebilirsiniz" bilgisi sunulur. İç temsil (private alan) ile dış arayüz (public metod) birbirinden ayrılır.

Erişilmesi istenen değişken varsa bunun için atama ve alma (setter / getter) fonksiyonları yazılır.

Her değişken için setter ve getter yazılmasına gerek yoktur. Eğer erişilmesi istenmeyen bir değişken varsa bu durumda ilgili fonksiyonlar eklenmez.

Sınıf ile ilgili işler yapan metotlar sınıfın içerisine taşınır.

Nesneyle ilgili tüm davranışlar sınıfın kendi bünyesinde olmalıdır. Bu sayede nesne "ne yapabileceğini" kendi içinde tanımlar; dışarıdaki kod sadece sonucu kullanır.

Erişim metotlarında çeşitli işlemler gerçekleştirilebilir. Örneğin yaş değeri 150'den büyük girildiğinde otomatik olarak yaş 0 olarak atanır. Ama genellikle erişim metotlarında koşul koyulması istenmez.

ERİŞİM DÜZEYLERİ

```
class kisi{
    private String ad;
    private int yas;
    private int no;
    public void setYas(int yas){
        if(yas<150)
            this.yas = yas;
        else
            yas = 0;
    }
    public int getYas(){
        return yas;
    }
    public int sonTikiHane(){
        return no % 100;
    }
}
```



```
public class Main {
    public static void main(String[] args){
        kisi k = new kisi();
        k.yas = 30;
    }
}
```



```
public class Main {
    public static void main(String[] args){
        kisi k = new kisi();
        k.setYas(30);
    }
}
```

Sınıfta private yaptığımız değişkenlere sınıf dışarısından erişemediğimiz için üstteki kod parçası hata verecektir.

Bu hata derleme zamanında (compile-time) yakalanır; yani program hiç çalışmaya başlamaz. nesne.marka = "BMW" gibi bir erişim denemesi, Java derleyicisi tarafından reddedilir.

ELEMAN FONKSİYONLAR (METOTLAR)

- Sınıf içerisinde
- Tüm elemanlara erişim
- Aşırı yüklenebilirler

Eleman fonksiyonlar sınıfın içerisinde bulunur ve sınıfın içerisinde oldukları için sınıfın tüm değişken ve diğer metotlarına erişebilirler.

Eleman fonksiyonlar bir nesne üzerinden çağrılır ve o nesnenin tüm alanlarına erişim yetkisine sahiptir — erişim belirleyiciden bağımsız olarak. private tanımlı bir alana sınıf dışından erişilemezken, aynı sınıfın bir eleman fonksiyonu o alana doğrudan ulaşabilir.

Eleman fonksiyonlar aşırı yüklenebilirler.

Aşırı yükleme (overloading), aynı isimde birden fazla metodun farklı parametre listeleriyle tanımlanabilmesidir. Hangi metodun çağrılacağına derleyici, geçilen

parametrelerin sayısı ve türüne bakarak karar verir. Örneğin `yazdir()`, `yazdir(String format)` ve `yazdir(String format, int ondalik)` metotları aynı sınıfta bir arada bulunabilir.

ELEMAN FONKSİYONLAR (METOTLAR)

```
class Kisi{
    public String ad;
    public int yas;
}

public class Main {
    public static void main(String[] args){
        Kisi k = new Kisi();
        k.ad = "Emir";
        k.yas = 253;
        System.out.println(k);
    }
}
```



```
class Kisi{
    private String ad;
    private int yas;
    private int no;
    public void setYas(int yas){
        if(yas<150) this.yas = yas;
        else yas = 0;
    }
    public int getKisi(){ return yas; }
    public int sonKisiHava(){ return no % 100; }
}

public class Main {
    public static void main(String[] args){
        Kisi k = new Kisi();
        System.out.println(k.yazdir());
    }
}
```

```
System.out.println(k.ad + " " + k.yas);
```

YAPICI FONKSİYONLAR

- Bildirim anında çağırılır
- İlk değer atama
- İsim
- Dönüş türü
- Aşırı yüklenebilirler

Yapıcı fonksiyonlar, sınıfın bir nesnesinin bildirimi yapıldığı anda (instance oluşturma) otomatik olarak çağırılan fonksiyonlardır. `new Araba()` çağırısı yapıldığı anda, derleyici önce heap bellekte nesne için yer ayırır, ardından ilgili constructor'ı otomatik olarak çağırır. Yapıcı fonksiyonlar ilk değer atamak ve nesnenin kullanılmasından önce gereken işlemleri yapmak için kullanılırlar.

Nesnenin tutarlı ve kullanıma hazır bir durumda başlaması önemlidir. Constructor olmadan bir nesne, alanları varsayılan değerlerde (null, 0, false) oluşur. Constructor bu riski ortadan kaldırır.

Yapıcı fonksiyonlar sınıfın ismi ile aynı olmalıdırlar. Değer döndürmezler. Dönüş türü (void) verilmez. Yapıcı fonksiyon içerisinde değer döndürmemelerinden ötürü return ifadesi kullanılmaz.

Bu kural Java ve C++ için geçerlidir. Dönüş türü yazılmaması constructor'ı sıradan bir metottan ayırt etmenin temel yoludur. Yanlışlıkla void yazılırsa derleyici bunu constructor değil farklı bir metot olarak yorumlar ve varsayılan constructor'ı ayrıca ekler.

Yapıcı fonksiyonlar da aşırı yüklenebilirler.

Farklı parametre kombinasyonlarıyla birden fazla constructor tanımlanabilir. Bu

YAPICI FONKSİYONLAR

```
class Kisi{
private String ad;
private int yas;
private int no;

public Kisi(){
}

public Kisi(String ad,int yas,int no){
this.ad = ad;
this.yas = yas;
this.no = no;
}

public void setYas(int yas){
}
```

```
public class Main {
    public static void main(String[] args){
        Kisi k = new Kisi();
        k.ad = "Emir";
        k.yas = 255;
        System.out.println(k);
    }
}
```

```
public class Main {
    public static void main(String[] args){
        Kisi k1 = new Kisi();
        Kisi k2 = new Kisi( ad: "Emir", yas: 255, no: 203022);
    }
}
```

YAPICI FONKSİYONLAR

```
class Kisi{
private String ad;
private int yas;
private int no;

public Kisi(){
}

public Kisi(String ad,int yas,int no){
this.ad = ad;
this.yas = yas;
this.no = no;
}

public void setYas(int yas){
}
```

```
class Kisi{
private:
std::string ad;
int yas;
int no;
public:
Kisi(){
}
Kisi(std::string ad,int yas, int no){
this->ad = ad;
this->yas = yas;
this->no = no;
}
```

```
class Kisi:
def __init__(self, ad, yas, no):
self._ad = ad
self._yas = yas
self._no = no

def son_iki_hane(self):
return self._no % 100
```

Yapıcı fonksiyonlar Python dışında sınıf ismi ile yazılır. Python'da ise init() ile tanımlanabilmektedir.

Java ve C++'ta constructor adı sınıf adıyla birebir aynıdır. Python'da ise init (çift alt çizgi-init-çift alt çizgi) adı sabit olup tüm sınıflar için aynıdır. Python'da bir sınıftan yalnızca bir init tanımlanabilir; birden fazla tanımlamak mümkün değildir. Sınıf değişkenlerine erişebilmek için this anahtar kelimesi kullanılır. Python'da ise self kullanılır. This anahtar kelimesi C++'ta pointer olarak kullanılır ve . yerine -> operandı ile erişim sağlanır.

Java'da this bir referanstır: this.marka = marka şeklinde nokta ile erişilir.

Python'da self de aynı şekilde referanstır ve self.marka = marka biçiminde kullanılır. C++'ta ise this bir pointer'dır; this->marka = marka şeklinde ok operatörüyle erişilir. this veya self kullanımının temel amacı, metot parametresiyle aynı ada sahip bir sınıf alanının belirsizliğini gidermektir: this.yas parametreyi değil, nesnenin kendi yas alanını işaret eder.

YIKICI FONKSİYONLAR

- Yaşam döngüsü
- Bellek iadesi
- Bitirilmesi gereken işlemler
 - Dosyalar
 - Veritabanları
 - Ağ bağlantıları
- Java için Non - deterministik
 - C++ için çağırılması zorunlu olmayan durumlar
 - Smart pointer (unique_ptr, shared_ptr)
- Aşırı yüklenemezler
- Değer almaz / döndürmezler

```
class Kisi{
private:
    std::string ad;
    int yas;
    int no;
public:
    Kisi(){
    }
    Kisi(std::string ad,int yas, int no){
        this->ad = ad;
        this->yas = yas;
        this->no = no;
    }
    ~Kisi(){
    }
}

int main(int argc, const char * argv[]) {
    Kisi *k1 = new Kisi();
    delete k1;
    std::unique_ptr<Kisi> k2 = std::make_unique<Kisi>();
    std::shared_ptr<Kisi> k3 = std::make_shared<Kisi>();
    return 0;
}
```

Bir nesnenin yaşam döngüsü tamamlandığında çalıştırılan metotlardır.

Nesnenin yaşam döngüsü sona erdiğinde — Java'da GC nesneyi topladığında, C++'ta ise nesne scope dışına çıktığında veya delete çağırıldığında — yıkıcı otomatik olarak çalışır. Programcının bunu manuel tetiklemesi gerekmez.

Nesnenin alınması ile kullanılan belleği iade etmek amacı ile kullanılır.

Yalnızca bellek değil; dosya tanıtıcıları, veritabanı bağlantıları, ağ soketleri ve diğer sistem kaynakları da yıkıcı içinde serbest bırakılabilir. Bu kaynaklar GC tarafından otomatik temizlenmez; yalnızca yıkıcı içinde açıkça kapatılırlarsa sistem kaynakları geri kazanılır.

Genellikle yıkıcılarla kullanılan dosyaların kapatılması, veritabanı bağlantılarının kesilmesi gibi işlemler gerçekleştirilirler.

Yıkıcılar (finalize) Java dilinde non-deterministik çalışırlar. Çağırıldıkları an bellek iadesi gerçekleşmez. Bu işlemi çöp toplayıcısı üstlenmektedir.

finalize() metodunun ne zaman çağırılacağı garanti altında değildir. GC bunu kendi iş yüküne ve bellek baskısına göre belirler. Hatta program normal şartlarda sonlanırsa finalize() hiç çağırılmayabilir. Java 9'dan itibaren finalize() deprecated olarak işaretlenmiştir.

C++'ta ise normalde new ile alınan değişkenin delete ile geri verilmesi gerekmektedir fakat yeni nesilde buna ihtiyaç bulunmaması için mekanizmalar

bulunmaktadır. Bunlar unique ve shared pointer yapılarıdır. Bu yapılar ile nesne oluşturulduğunda delete işleminin yapılmasına ihtiyaç kalmaz. Unique pointer en hızlı alternatiftir ve kopyalanamaz. Shared pointer ise kopyalanmaya izin verir ama unique pointer'dan performans açısından daha yavaştır.

unique_ptr tek sahiplik (single ownership) modeli uygular: yalnızca bir pointer nesneye sahip olabilir, kopyalanamaz, yalnızca taşınabilir (move semantics). Nesnenin scope'u bittiğinde destructor otomatik çağrılır; delete yazmaya gerek kalmaz. shared_ptr ise referans sayacı (reference counting) kullanır: kaç pointer nesneye sahipse sayaç o kadardır. Son sahip scope dışına çıktığında sayaç sıfıra iner ve nesne otomatik silinir. Ancak sayaç güncellemesi thread-safe olduğundan unique_ptr'dan daha yavaştır. Modern C++'ta new/delete doğrudan yazmak yerine make_unique<T>() ve make_shared<T>() fonksiyonları tercih edilir.

ÇÖP TOPLAYICI (GARBAGE COLLECTOR)

- Garbage Collector
- Heap
- Nesiller
 - Eden
 - Survivor
 - Tenured
- Permanent - Metaspace

*<https://medium.com/@tugrulbayrak/jvm-garbage-collector-nedir-96e76b6f6239>

*<https://stackoverflow.com/questions/2129044/java-heap-terminology-young-old-and-permanent-generations>

GC (Garbage Collector), bellekten alınan nesnelerin iadesi görevini üstlenir. Java'da programcı new ile nesne oluşturur; ancak delete veya free gibi bir çağrı yapmaz. GC arka planda sürekli çalışarak artık kullanılmayan — yani kendisine ulaşan hiçbir referans kalmayan — nesneleri tespit eder ve bellekten kaldırır. Bu sayede bellek sızıntısı riski büyük ölçüde azalır.

GC, heap alanına bakıp kullanılmayan nesnelerin silinmesi prensibine göre çalışır.

Java'da heap nesillere bölünmüştür.

Bu nesil tabanlı (generational) yaklaşımın arkasındaki fikir şudur: uygulamaların

büyük çoğunluğunda nesnelerin büyük kısmı kısa ömürlüdür (bir döngü iterasyonunda oluşturulup hemen terk edilir). Kısa ömürlü nesneleri küçük ve hızlı temizlenen bir alanda tutmak, tüm heap'i sürekli taramaktan çok daha verimlidir.

Eden space: İlk oluşturulan nesneler buraya yerleştirilir. Daha sonra burası dolduğunda Minor GC çalışır ve kullanılmayanları temizler.

Minor GC çok hızlıdır ve uygulamayı yalnızca kısa süreliğine durdurur. Eden'da referansı kalan nesneler Survivor Space'e taşınır; geri kalanlar silinir.

Survivor space: Eden'da silinmeyen nesneler buraya yerleşir.
